

TP1 Bitácora de proceso con IA 2026

Comisión Matias

Estudiantes: Cardozo, Meza, Bagnera, Alvarez y Ayala.

Tecnologías utilizadas

- Entorno / Plataforma de trabajo

ChatGPT, Codex.

- Modelo de IA utilizado

GPT-5.5

Entrada #1

1. Encabezado (una línea)

Entrada 1 — [12/5/2026] — [Centrar los trazos]

2. Objetivo (1-2 oraciones)

Queríamos que los trazos no aparezcan fuera de la pantalla, con un margen.

3. Prompt (textual)

Quiero que no aparezcan trazos fuera de la pantalla, solo en el centro, pone márgenes.

4. Resultado visual



```
let x = random(width * 0.3, width * 0.7);  
let y = random(height * 0.3, height * 0.7);
```

5. Evaluación y corrección (2-5 oraciones)

Nos pareció correcto porque si bien las líneas pueden llegar a salirse del margen, no es que aparecen por fuera de este y no se ven.

6. Aprendizaje (1-2 oraciones)

Aprendimos que esa multiplicación sirve para sacar un porcentaje del tamaño del canvas. Por ejemplo `width * 0.3` significa el 30% de ancho, por lo que el código dice es: Genera un número aleatorio entre el 30% y el 70% del ancho del canvas.

Entrada #2

1. Encabezado (una línea)

Entrada 2 — [12/5/2026] — [Borrar los trazos]

2. Objetivo (1-2 oraciones)

Queríamos poder borrar los trazos desde el último creado hasta el primero.

3. Prompt (textual)

Quiero que con la `r` se borren desde el último creado al primero, no uses `let of`, mantente con arreglos y variables.

4. Resultado visual

```
let indiceDibujo = [];  
let xDibujo = [];  
let yDibujo = [];  
  
let cantidadDibujos = 0;
```

```
image(  
  trazos[indiceDibujo[i]],  
  xDibujo[i],  
  yDibujo[i]  
);  
  
}  
  
function mousePressed() {  
  
  indiceDibujo[cantidadDibujos] = int(random(1, cantidadTrazos + 1));  
  
  xDibujo[cantidadDibujos] = random(width * 0.3, width * 0.7);  
  
  yDibujo[cantidadDibujos] = random(height * 0.3, height * 0.7);  
  
  cantidadDibujos++;  
  
}  
  
function keyPressed() {  
  
  if (key == "r" || key == "R") {  
  
    if (cantidadDibujos > 0) {  
  
      cantidadDibujos--;  
  
    }  
  
  }  
  
}
```

5. Evaluación y corrección (2-5 oraciones)

Nos pareció correcto que en el código use if y else para borrar los trazos o dibujarlos según la tecla r o el click.

6. Aprendizaje (1-2 oraciones)

Aprendimos que a la ia hay que especificarle muy bien con que puede o no trabajar, acá directamente no le permitimos usar ciertas funciones y así hizo un código mucho más prolijo.

Entrada #3

1. Encabezado (una línea)

Entrada 3 — [12/5/2026] — [efecto corte falla]

2. Objetivo (1-2 oraciones)

Queremos generar el efecto corte, o sea que el trazo aparezca de arriba hacia abajo cuando se hace click.

3. Prompt (textual)

Quiero que los trazos se dibujen, no que aparezcan de una sino que parezca que se van dibujando en el lienzo.

4. Resultado visual

```
image(  
    trazos[indiceDibujo[i]],  
    xDibujo[i],  
    yDibujo[i],  
    tamañoDibujo[i],  
    tamañoDibujo[i]  
);  
  
if (tamañoDibujo[i] < 300) {  
  
    tamañoDibujo[i] += 8;  
  
}  
  
}
```

```
tamañoDibujo[cantidadDibujos] = 0;
```

5. Evaluación y corrección (2-5 oraciones)

No nos sirvió esto, ya que en vez de hacer efecto corte y hacer que el PNG aparezca de una esquina a otra lo que sucedió fue que el trazo aparece agrandandose. así que le tuve que especificar esto a la ia “no me sirve yo quiero que vaya apareciendo de arriba a abajo el png tipo corte no se si entiendes”

6. Aprendizaje (1-2 oraciones)

Para el sistema visual del artista necesitamos, si o si un efecto de corte para el trazo, ya que si no, los trazos parecen figuras que se agrandan y no respetan la obra.

Entrada #4

1. Encabezado (una línea)

Entrada 4 — [12/5/2026] — [Corte logrado]

2. Objetivo (1-2 oraciones)

lograr que el png del trazo se dibuje de una esquina a otra rápido para que se dé la sensación de un corte.

3. Prompt (textual)

Pulilo más, hay varios errores, primero que antes de dibujarse el trazo aparece grande por milisegundos, segundo que sigue sin parecer un corte es como que se acomoda, y tercero, no me gusta el código que haces, quiero calidad, un código con variables y funciones, también puedes usar if, else, ciclo For, etc, podrías arreglar eso.

4. Resultado visual

```
function draw() {  
  
  background(255);  
  
  for (let i = 0; i < cantidadDibujos; i++) {  
  
    let numeroTrazo = indiceDibujo[i];  
  
    let imagenTrazo = trazos[numeroTrazo];  
  
    let anchoTrazo = imagenTrazo.width;  
  
    let altoTrazo = imagenTrazo.height;  
  
    let posicionX =xDibujo[i];  
  
    let posicionY = yDibujo[i];  
  
    let alturaActual = altoVisible[i];
```

```

copy(

    imagenTrazo,

    0,
    0,

    anchoTrazo,
    alturaActual,

    posicionX - anchoTrazo / 2,
    posicionY - altoTrazo / 2,

    anchoTrazo,
    alturaActual

);

if (alturaActual < altoTrazo) {

    altoVisible[i] = altoVisible[i] + 12;

}

else {

    altoVisible[i] = altoTrazo;

}

}

}

```

5. Evaluación y corrección (2-5 oraciones)

Nos gusto como quedo el efecto de corte, lo que sí estaba muy lento, por lo que aumentamos la velocidad modificando esta línea de código

```
altoVisible[i] = altoVisible[i] + 12;
```

a +110.

6. Aprendizaje (1-2 oraciones)

La ia utilizó una función llamada copy(), que copia un pedazo de una imagen. Se suele usar para apariciones como la que queríamos hacer con los trazos.

Entrada #5

1. Encabezado (una línea)

Entrada #5 — [12/05/26] — [Cambio de color del fondo]

2. Objetivo (1-2 oraciones)

Queríamos lograr la interacción de que al presionar una tecla cambie la imagen de fondo por otra de las que teníamos dentro de la carpeta “fondos”.

3. Prompt (textual)

Si quiero que al presionar una tecla el fondo cambie de color a otro de la carpeta fondos como hago?

4. Resultado visual

```
let fondos = [];  
let fondoActual = 0;  
  
function preload() {  
  fondos[0] = loadImage("fondos/fondo1.png");  
  fondos[1] = loadImage("fondos/fondo2.png");  
  fondos[2] = loadImage("fondos/fondo3.png");  
}  
  
function setup() {  
  createCanvas(800, 600);  
}  
  
function draw() {  
  image(fondos[fondoActual], 0, 0, width, height);  
}  
  
function keyPressed() {  
  if (key === ' ') {  
    fondoActual++;  
  
    if (fondoActual >= fondos.length) {  
      fondoActual = 0;  
    }  
  }  
}
```



5. Evaluación y corrección (2-5 oraciones)

Nos sirvió el funcionamiento, pero no nos gustó como estaba redactado el código en la parte de arreglos cargados uno por uno, eso lo cambiamos por un for, para que directamente tome los fondos desde ahí.

6. Aprendizaje (1-2 oraciones)

Acá decidimos directamente cargar fondos ya que el artista usa degradados con efectos de pintura, para eso borramos en las pinturas los trazos con ia. En todo caso aprendimos a revisar bien que hace la ia para que no escriba código innecesario.

Entrada #6

1. Encabezado (una línea)

Entrada #6 — [8/06/26] — [revisar y corregir proyecto]

2. Objetivo (1-2 oraciones)

Corregir los errores que encontramos en el código.

3. Prompt (textual)

El proyecto ya abre en Live Server y no tira errores críticos, pero no detecta bien la voz o no veo cambios cuando hablo.

Quiero que primero hagas un diagnóstico del problema y luego propongas una versión corregida del código.

Muy importante:

Incluí instrucciones para probar si el micrófono está funcionando, por ejemplo:

- * qué valores debería ver en amplitud
- * cómo saber si el navegador tomó el micrófono
- * qué hacer si amplitud queda siempre en 0
- * qué umbrales iniciales probar
- * cómo calibrar los valores según mi ambiente

4. Resultado visual

```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<title>Resultado Visual</title>

<style>
body{
  margin:0;
  display:flex;
  justify-content:center;
  align-items:center;
  height:100vh;
  background:#eaeaea;
}

.canvas{
  position:relative;
  width:720px;
  height:1080px;
  overflow:hidden;

  background:
    radial-gradient(circle at
    center,#d8c7a4,#9f8765,#5c4b3f);
}

/* trazos */

.trazo{
  position:absolute;
  border-radius:100px;
  opacity:.45;
}

.t1{
  width:20px;
  height:280px;
  background:black;
  left:320px;
  top:220px;
  transform:rotate(-15deg);
}

.t2{
  width:18px;
  height:220px;
  background:black;
  left:390px;
  top:350px;
  transform:rotate(12deg);
}
```

```
.t3{
  width:25px;
  height:310px;
  background:black;
  left:250px;
  top:420px;
  transform:rotate(-30deg);
}

.t4{
  width:15px;
  height:180px;
  background:black;
  left:460px;
  top:520px;
  transform:rotate(20deg);
}

.t5{
  width:30px;
  height:250px;
  background:black;
  left:350px;
  top:620px;
  transform:rotate(8deg);
}

/* panel debug */

.debug{
  position:absolute;
  top:16px;
  left:16px;

  width:320px;
  padding:15px;

  background:rgba(0,0,0,.7);
  color:white;

  font-family:monospace;
  font-size:14px;
}

.barra{
  margin-top:10px;
  height:10px;
  width:180px;
  background:#78dcff;
}
</style>
</head>

<body>

<div class="canvas">
```

```
<div class="trazo t1"></div>
<div class="trazo t2"></div>
<div class="trazo t3"></div>
<div class="trazo t4"></div>
<div class="trazo t5"></div>

<div class="debug">
  click o tecla A: activar audio<br><br>

  mic activo: true<br>
  amplitud: 0.0345<br>
  graves: 82<br>
  medios: 65<br>
  agudos: 110<br>
  duración: 1850 ms<br>
  estado: agudo fuerte<br>

  <div class="barra"></div>
</div>

</div>

</body>
</html>
```

5. Evaluación y corrección (2-5 oraciones)

Se detectaron problemas en la activación del micrófono y en detectar el sonido, lo que impedía una respuesta visual consistente. Además, la composición se saturaba por la acumulación de trazos. Para corregirlo, se reorganizó la estructura del código, se ajustaron

los umbrales de audio y se incorporó un sistema de depuración que permitió mejorar la interacción entre sonido y visualización.

6. Aprendizaje (1-2 oraciones)

Aprendimos la importancia de estructurar el código de manera organizada para facilitar las futuras modificaciones que realicemos.

Entrada #7

1. Encabezado (una línea)

Entrada #7 — [9/06/26] — [que aparezcan los trazos PNG]

2. Objetivo (1-2 oraciones)

Detectar por qué los trazos en PNG no aparecen, corregir el error.

3. Prompt (textual)

Necesito que hagas esto:

1. Reemplazar `copy()` por `image()` con esta estructura:

```
image(  
  imagenTrazo,  
  -anchoTrazo / 2,  
  -altoTrazo / 2,  
  anchoTrazo,  
  alturaActual,  
  0,  
  0,  
  imagenTrazo.width,  
  alturaActual / dibujo.escala  
);
```

2. Verificar que los trazos se dibujen después del fondo.

3. Verificar que `dibujarTrazos()` se llame dentro de `draw()`.

4. Verificar que `agregarTrazo()` realmente haga dibujos.`push(...)`.

5. Agregar una tecla de prueba T que ejecute `agregarTrazo("fino")`, para testear los trazos sin depender del audio.

6. Agregar logs temporales:

- cantidad de dibujos

- índice de trazo
- ancho y alto del trazo
- opacidad
- escala

7. Revisar si los umbrales del audio están haciendo que nunca se llame a agregarTrazo().
8. Mantener la estructura semántica del código ordenada.

No quiero que cambies toda la obra. Quiero que corrijas específicamente por qué los trazos PNG no aparecen.

4. Resultado visual

<pre> <!DOCTYPE html> <html lang="es"> <head> <meta charset="UTF-8"> <title>Prueba de Trazos Corregidos</title> <style> body{ margin:0; background:#ddd; display:flex; justify-content:center; align-items:center; height:100vh; } .canvas{ position:relative; width:720px; height:1080px; background:linear- gradient(180deg,#cbb89d,#a48b6a,#6f5d4e); overflow:hidden; } /* trazos visibles */ .trazo{ position:absolute; background:black; border-radius:100px; opacity:.45; } .trazo1{ width:18px; height:260px; left:330px; top:260px; transform:rotate(-12deg); } .trazo2{ width:22px; height:320px; left:390px; top:350px; transform:rotate(18deg); } .trazo3{ width:16px; height:220px; left:280px; top:500px; transform:rotate(-28deg); } .debug{ </pre>	<pre> .debug{ position:absolute; top:16px; left:16px; width:330px; padding:15px; background:rgba(0,0,0,.75); color:white; font-family:monospace; font-size:14px; } .barra{ width:120px; height:10px; margin-top:8px; background:#78dcff; } </style> </head> <body> <div class="canvas"> <div class="trazo trazo1"></div> <div class="trazo trazo2"></div> <div class="trazo trazo3"></div> <div class="debug"> tecla T: agregarTrazo("fino")

 cantidad dibujos: 3
 índice: 12
 opacidad: 82
 escala: 0.54

 amplitud: 0.008
 estado: silencio

 umbralSonido: 0.010
 umbralAlto: 0.045
 <div class="barra"></div> </div> </div> </body> </html> </pre>
--	--

5. Evaluación y corrección (2-5 oraciones)

Se identificó que el sistema de audio funcionaba correctamente, pero los trazos PNG no se visualizaban debido a un problema en la lógica de renderizado. La corrección consistió en revisar la forma en que se dibujaban las imágenes, incorporar herramientas de prueba y depuración, y verificar que los trazos se generaran y mostraran correctamente en pantalla.

6. Aprendizaje (1-2 oraciones)

Aprendimos que cuando un sistema presenta errores visuales es importante aislar cada parte del proceso para identificar el origen del problemas.

Entrada #8

1. Encabezado (una línea)

Entrada #8 — [9/06/26] — [mejora conceptual y técnica de la obra]

2. Objetivo (1-2 oraciones)

Implementar cambios para que la obra quede más coherente.

3. Prompt (textual)

Quiero que la composición visual se acerque más a la lógica compositiva de las pinturas de referencia , donde los trazos se concentran en zonas de energía y generan una masa visual coherente.

Problema actual:

La composición se ve demasiado dispersa y aleatoria.

Cada nuevo trazo aparece en una posición independiente, lo que genera ruido visual y no respeta la lógica de acumulación presente en las obras de referencia.

Quiero que implementes tres cambios principales:

1. Núcleo compositivo

Crear un sistema de núcleo de composición:

```
js
let nucleoX;
let nucleoY;
```

El objetivo es que los trazos no aparezcan distribuidos uniformemente por toda la pantalla.

Deben concentrarse alrededor de una zona dominante.

Necesito que propongas dónde conviene inicializar:

* nucleoX

- * nucleoY

y en qué función deberían actualizarse.

Además quiero que me expliques:

- * por qué elegiste esa ubicación
- * cómo afecta la composición
- * cómo se relaciona con las pinturas de referencia

2. Distribución gaussiana alrededor del núcleo

Actualmente el código utiliza algo similar a:

```
js
x = random(...)
y = random(...)
```

Quiero reemplazarlo por una distribución gaussiana utilizando:

```
js
randomGaussian()
```

Objetivo:

- * Mayor densidad cerca del núcleo.
- * Menor densidad a medida que nos alejamos.
- * Evitar una distribución uniforme.

Quiero que determines:

- * desviación estándar recomendada para X
- * desviación estándar recomendada para Y

para un canvas vertical de:

```
js
720 x 1080
```

También quiero que analices:

- * qué valores generan una composición demasiado cerrada
- * qué valores generan una composición demasiado dispersa

y me propongas rangos razonables.

3. Dirección dominante de los trazos

Actualmente cada trazo puede ser:

- * vertical
- * horizontal
- * curva

pero no existe una dirección compositiva general.

Quiero incorporar:

```
js  
let anguloDominante;
```

Objetivo:

Que cada composición tenga una dirección visual predominante, igual que ocurre en las pinturas de referencia.

Ejemplos:

- * vertical dominante
- * diagonal ascendente
- * diagonal descendente
- * horizontal dominante

Necesito que propongas:

- * cómo inicializar ese ángulo
- * cómo modificarlo según el estado sonoro
- * cómo hacer que cada nuevo trazo tenga pequeñas variaciones alrededor de ese ángulo

Relación con el sonido

Quiero que estos tres sistemas también dialoguen con el audio.

Proponé un comportamiento razonable como:

- * Sonidos graves → desplazan el núcleo hacia abajo.
- * Sonidos agudos → desplazan el núcleo hacia arriba.
- * Sonidos sostenidos → expanden el radio de dispersión.
- * Silencio → vuelve lentamente al centro.

No quiero movimientos exagerados.

La obra debe seguir pareciendo una pintura abstracta y no un visualizador musical.

Restricciones

- * Mantener los assets actuales.
- * Mantener los PNG de trazos.
- * Mantener la estructura general del proyecto.
- * No reescribir toda la aplicación.
- * Modificar únicamente las partes necesarias.
- * Explicar exactamente en qué funciones debe ir cada cambio:

- * variables globales
- * setup()
- * actualizarObra()
- * agregarTrazo()
- * dibujarTrazos()

Resultado esperado

La composición debe dejar de parecer una lluvia de trazos aleatorios y empezar a comportarse como una acumulación de energía visual alrededor de un núcleo, respetando mucho más la lógica compositiva de las obras de referencia.

4. Resultado visual

```
<div class="canvas">
  <div class="trazo t1"></div>
  <div class="trazo t2"></div>
  <div class="trazo t3"></div>
  <div class="trazo t4"></div>
  <div class="trazo t5"></div>
  <div class="trazo t6"></div>
</div>

<style>
.canvas{
  width:720px;
  height:1080px;
  position:relative;
  background:#c8b79e;
  overflow:hidden;
}

.trazo{
  position:absolute;
  background:black;
  opacity:.45;
  border-radius:100px;
}

.t1{left:320px; top:320px; width:18px;
height:260px; transform:rotate(22deg);}
.t2{left:360px; top:400px; width:22px;
height:300px; transform:rotate(28deg);}
.t3{left:280px; top:470px; width:16px;
height:220px; transform:rotate(18deg);}
.t4{left:410px; top:500px; width:20px;
height:280px; transform:rotate(25deg);}
.t5{left:340px; top:600px; width:18px;
height:240px; transform:rotate(30deg);}
.t6{left:300px; top:560px; width:14px;
height:180px; transform:rotate(20deg);}
</style>
```


5. Evaluación y corrección (2-5 oraciones)

Se observó que la composición resultaba demasiado dispersa y aleatoria, dificultando que se perciba como una estructura visual coherente. Para corregirlo, se incorporó un núcleo de energía que concentra los trazos, una dirección dominante para organizar la composición y una relación más sutil entre las variaciones sonoras y la distribución de los elementos visuales.

6. Aprendizaje (1-2 oraciones)

Aprendimos que la distribución de los elementos visuales influye significativamente en la lectura de una composición. También que pequeños ajustes en la posición, orientación y agrupación de los trazos pueden generar resultados más equilibrados y coherentes con la intención estética de la obra.

Entrada #9

1. Encabezado (una línea)

Entrada #9 — [10/06/26] — [reconocimiento de sonido no tonal]

2. Objetivo (1-2 oraciones)

Corregir el umbral de audio que aún sigue siendo muy alto, y que el código pueda reconocer el sonido no tonal lo suficientemente fuerte para que se cumpla con la condición de borrar las líneas.

3. Prompt (textual)

Contexto:

La obra ya funciona con micrófono, fondos y trazos PNG. El sistema detecta voz, cambia fondos y dibuja trazos. Pero todavía hay problemas importantes:

1. El umbral de audio sigue siendo demasiado alto.
2. El sonido no tonal casi nunca se reconoce con suficiente fuerza como para borrar líneas.
3. La distribución gaussiana y el núcleo compositivo parecen estar implementados, pero visualmente los trazos todavía se sienten demasiado aleatorios.
4. Necesito que la obra funcione para alguien que nunca la usó, con instrucciones simples de lógica vocal/tonal.
5. Necesito que los trazos se concentren en zonas energéticas de la obra, no que aparezcan dispersos por todo el canvas.
6. Quiero evitar que las líneas toquen demasiado los laterales.
7. Quiero que la composición se concentre más en la zona media y pueda cortar más naturalmente en bordes superior e inferior.

8. Quiero incorporar variación de opacidad: algunos trazos al 90%, otros al 50%, por ejemplo.

Objetivo general:

La obra debe dejar de sentirse como una lluvia de trazos aleatorios y comportarse como una acumulación de energía visual alrededor de un núcleo compositivo.

Necesito que analices si el problema viene de:

- * umbrales de sonido demasiado altos
- * mala detección del sonido no tonal
- * mala implementación de nucleoX y nucleoY
- * desviaciones gaussianas demasiado grandes
- * falta de constrain()
- * posiciones demasiado dispersas
- * lógica visual demasiado aleatoria

Quiero que revises especialmente estas partes:

- * variables globales de audio
- * analizarAudio()
- * actualizarEstadoSonoro()
- * actualizarObra()
- * agregarTrazo()
- * lógica de nucleoX y nucleoY
- * distribución con randomGaussian()
- * opacidad de los trazos
- * límites laterales del canvas

4. Resultado visual

```
<div class="canvas">

  <div class="trazo principal t1"></div>
  <div class="trazo secundario t2"></div>
  <div class="trazo principal t3"></div>
  <div class="trazo secundario t4"></div>
  <div class="trazo principal t5"></div>
  <div class="trazo secundario t6"></div>

  <div class="debug">
    amp: 0.034<br>
    amp suave: 0.029<br>
    graves: 52<br>
    medios: 61<br>
    agudos: 89<br>
    estado: agudo fuerte<br>
    trazos: 37
  </div>

</div>

<style>
.canvas{
  width:720px;
  height:1080px;
  position:relative;
  background:#cdbda4;
  overflow:hidden;
}

.trazo{
  position:absolute;
  background:black;
  border-radius:100px;
}

.principal{
  opacity:.85;
}

.secundario{
  opacity:.45;
}

.t1{left:330px; top:290px; width:18px;
height:290px; transform:rotate(18deg);}
.t2{left:380px; top:370px; width:15px;
height:220px; transform:rotate(24deg);}
.t3{left:300px; top:440px; width:20px;
height:330px; transform:rotate(15deg);}
.t4{left:395px; top:500px; width:14px;
height:180px; transform:rotate(28deg);}
.t5{left:345px; top:570px; width:22px;
height:260px; transform:rotate(20deg);}
.t6{left:285px; top:620px; width:12px;
height:170px; transform:rotate(12deg);}
```

```
.debug{
  position:absolute;
  top:15px;
  left:15px;
  padding:12px;
  background:rgba(0,0,0,.75);
  color:white;
  font-family:monospace;
  font-size:13px;
}
</style>
```

5. Evaluación y corrección (2-5 oraciones)

Se observó que la detección de sonidos no tonales era demasiado limitada y que muchas veces la obra no reaccionaba correctamente al estímulo “sh”. Para corregirlo, se ajustaron los umbrales de audio, se incorporó una amplitud suavizada y se mejoró la lógica de reconocimiento sonoro, obteniendo una respuesta más estable y precisa.

6. Aprendizaje (1-2 oraciones)

Aprendimos que reconocer distintos tipos de sonido requiere ajustar cuidadosamente los parámetros de análisis de audio. A partir de esta corrección, logramos una mejor detección de sonidos no tonales, permitiendo que la obra responda de manera más precisa y coherente a diferentes estímulos sonoros.